

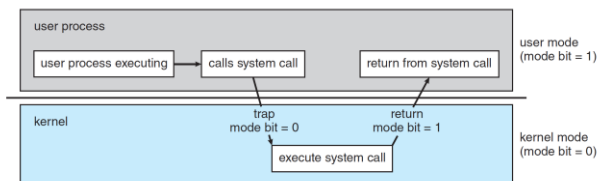
KERNEL LINUX

Jorge Estrela da Silva

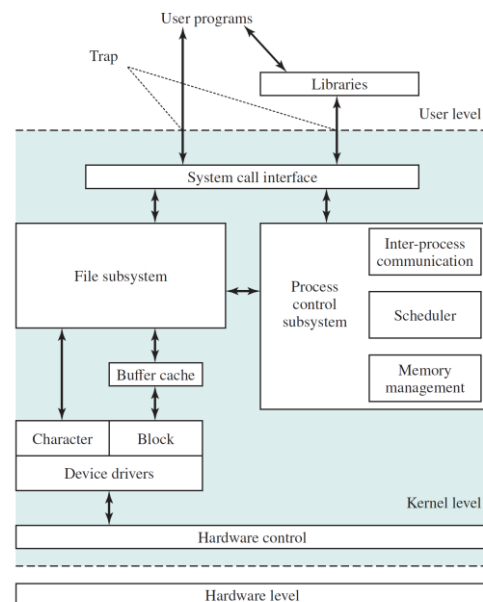
1

ARQUITETURA TÍPICA DE SISTEMAS LINUX

- User space (“userland”) vs kernel space
 - User level does not allow direct access to “hardware” (main memory, I/O memory, interrupts)
 - User programs are run at user level



Jorge Estrela da Silva



2

2

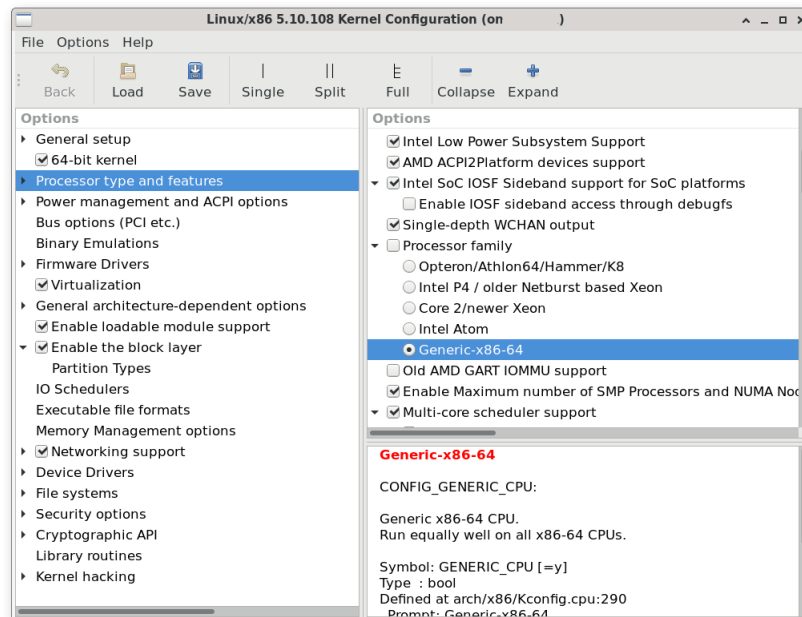
LINUX: KERNEL MODULAR

- **Kernel Linux: altamente configurável**
 - Possível empacotar todas as funcionalidades selecionadas num único ficheiro.
 - Tipo de sistema de ficheiros suportados
 - Periféricos a suportar (câmaras, impressoras, etc.)
 - Módulos/interfaces de E/S (controladores USB)
 - Protocolos de rede suportados (IPv4, IPv6, etc.)
 - Etc.
 - Alternativamente, é possível ter algumas funcionalidades na forma de **módulos** (Linux Kernel Modules – LKM) que são carregados conforme necessário
 - Não faz sentido gerar um ficheiro de kernel com todas as funcionalidades possíveis
 - Requisitos de memória seriam elevados
 - Não existe nenhum sistema que use todos os dispositivos suportados pelo Linux

Jorge Estrela da Silva

3

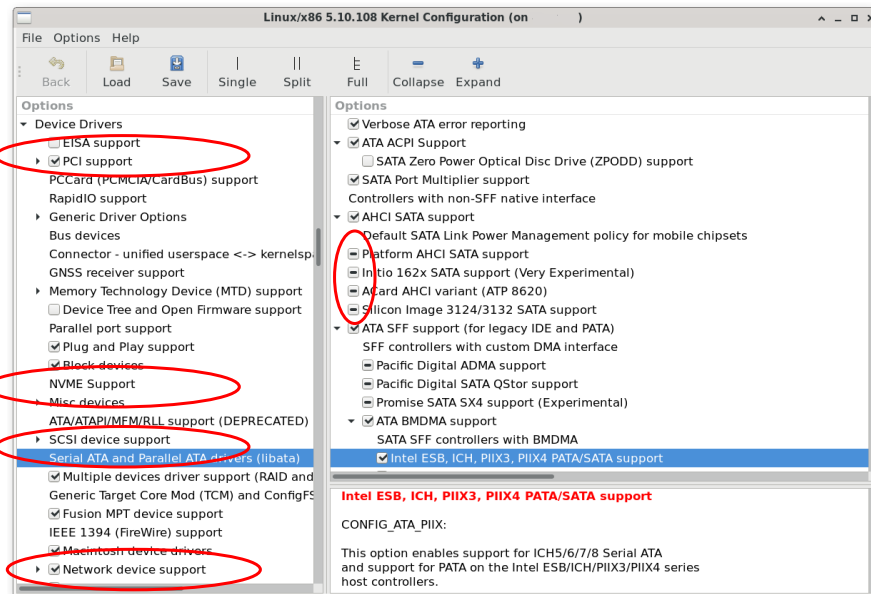
3



Jorge Estrela da Silva

4

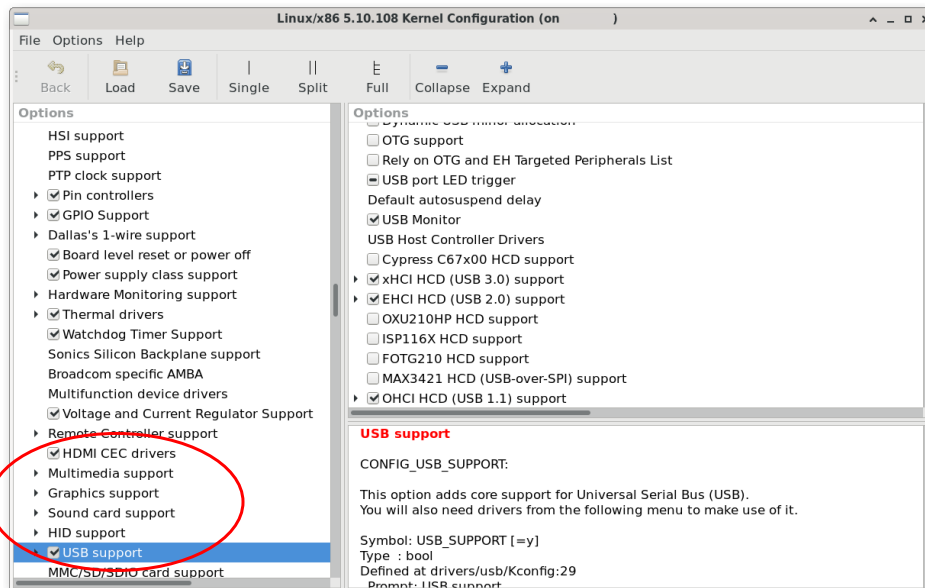
4



Jorge Estrela da Silva

5

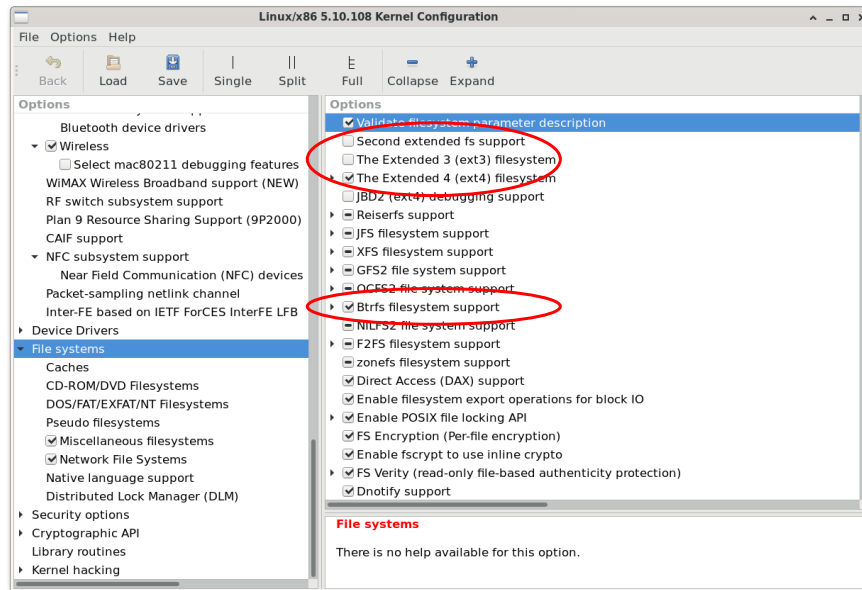
5



Jorge Estrela da Silva

6

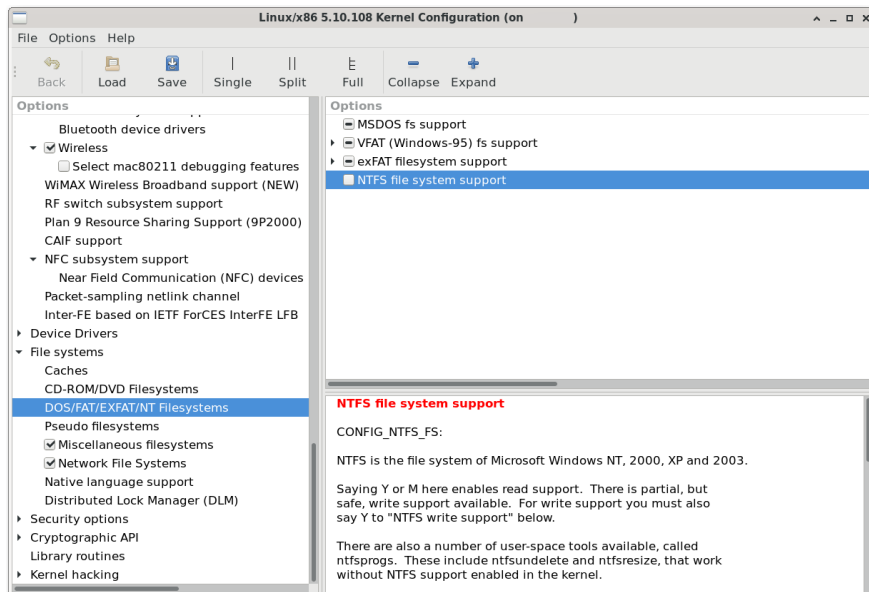
6



Jorge Estrella da Silva

7

7



Jorge Estrella da Silva

8

8

CARREGAMENTO DOS MÓDULOS DE KERNEL

- Tradicionalmente, os módulos de *kernel* são compilados para ficheiros com a extensão “.ko”
 - As versões mais recentes do kernel suportam compressão dos módulos (e.g. “mymod.ko.xz”)
- Alguns módulos fazem chamadas a funções definidas noutros módulos
 - Se um módulo X usa funções de Y, diz-se que Y é uma dependência de X
- Carregamento manual de módulos:
 - **modprobe** – carregamento automatizado, incluindo dependências
 - Baseado no ficheiro de dependências /lib/modules/<kernel release>/modules.dep
 - Criado pelo comando **depmod**, tipicamente na altura da geração do *kernel*
 - **insmod** – carregamento de módulo individual.
 - Obriga a especificar caminho e nome completo do módulo
 - Dependências têm que ser carregadas previamente

Jorge Estrela da Silva



9

LISTAGEM DE MÓDULOS CARREGADOS

- Comando **lsmod**

```
[user@pc]$ lsmod
```

Module	Size	Used by
vfat	20480	0
fat	86016	1 vfat
vhost_net	36864	0
vhost	53248	1 vhost_net

- Menos amigável: **cat /proc/modules**

Jorge Estrela da Silva



10

```
[user@pc]$ cd /lib/modules/$(uname -r)
[user@pc 5.16.16-200.fc35.x86_64]$ find . -name "*.ko.xz"|grep net|grep intel
./kernel/drivers/net/phy/intel-xway.ko.xz
./kernel/drivers/net/ethernet/stmicro/stmmac/dwmac-intel.ko.xz
./kernel/drivers/net/ethernet/intel/e1000/e1000.ko.xz
./kernel/drivers/net/ethernet/intel/iavf/iavf.ko.xz
./kernel/drivers/net/ethernet/intel/e1000e/e1000e.ko.xz
./kernel/drivers/net/ethernet/intel/e100.ko.xz
./kernel/drivers/net/ethernet/intel/ixgbe/ixgbe.ko.xz
./kernel/drivers/net/ethernet/intel/i40e/i40e.ko.xz
./kernel/drivers/net/ethernet/intel/igc/igc.ko.xz
./kernel/drivers/net/ethernet/intel/ixgbev/ixgbev.ko.xz
./kernel/drivers/net/ethernet/intel/igbvf/igbvf.ko.xz
./kernel/drivers/net/ethernet/intel/fm10k/fm10k.ko.xz
./kernel/drivers/net/ethernet/intel/igb/igb.ko.xz
./kernel/drivers/net/ethernet/intel/ice/ice.ko.xz
./kernel/drivers/net/ethernet/intel/ixgb/ixgb.ko.xz
./kernel/drivers/net/wireless/intel/iwlwifi/dvm/iwldvm.ko.xz
./kernel/drivers/net/wireless/intel/iwlwifi/iwlwifi.ko.xz
./kernel/drivers/net/wireless/intel/iwlwifi/mvm/iwlmvm.ko.xz
./kernel/drivers/net/wireless/intel/iwlegacy/iwlegacy.ko.xz
./kernel/drivers/net/wireless/intel/iwlegacy/iwl3945.ko.xz
./kernel/drivers/net/wireless/intel/iwlegacy/iwl4965.ko.xz
```

Jorge Estrela da Silva

11

11

ESQUELETO MÍNIMO DE UM LKM

- **init_module:** executada quando módulo é carregado
 - Deverá retornar valor negativo em caso de erro, 0 no caso de sucesso
 - **cleanup_module:** executada quando módulo é descarregado
 - Tipo de licença de utilização: MODULE_LICENSE()
 - GPL, GPL v2, GPL and additional rights, Dual MIT/GPL, Dual BSD/GPL, Dual MPL/GPL, Proprietary
 - <https://www.kernel.org/doc/html/v5.10/process/license-rules.html>
- ```
#include <linux/module.h>
MODULE_LICENSE("GPL");

int init_module(void){
 return 0;
}

void cleanup_module(void){
}
```
- Possível redefinir o nome destas funções (*macros* module\_init/module\_exit)

Jorge Estrela da Silva

12

12

# DEVICE DRIVERS EM LINUX

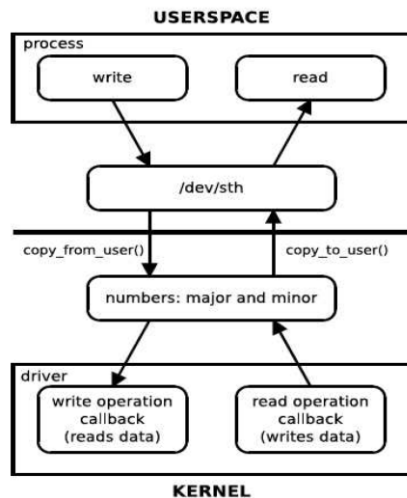
- Módulos destinados a estabelecer uma “ponte” entre dispositivos físicos e os programas dos utilizadores.
  - Dispositivo físico é abstraído na forma de um ficheiro
- Dois tipos principais de *device drivers*:
  - Orientados ao bloco (leitura em blocos de dados, permite acesso aleatório)
  - Orientados ao carácter (acesso aos dados é sequencial)
- Cada tipo de *device driver* obedece a uma API específica.
- Interface entre os módulos e os processos do espaço do utilizador é feita através dos ficheiros de dispositivo, presentes em **/dev**. Exemplos:
  - `crw-rw----. 1 root dialout 4, 64 mar 23 00:55 ttyS0`
  - `brw-rw----. 1 root disk 8, 0 mar 23 21:44 sda`
- Não esquecer que o código dos módulos é executado em *kernel mode*
  - Privilégios máximos
  - Qualquer erro no código pode comprometer a estabilidade do sistema

Jorge Estrela da Silva

13

13

# CHARACTER DEVICES



- `dev_t` – valor de 32 bits para registo/identificação do device driver
  - *major number* de 12 bits + 20 bits para *minor number*
- Duas opções para fazer o registo:
  - Com *major number* pré-definido:
    - `int register_chrdev_region(dev_t first, unsigned int count, char *name);`
      - *count* - número de subdispositivos geridos por este módulo
  - Deixar a escolha do *major number* para o kernel:
    - `int alloc_chrdev_region(dev_t* dev, unsigned baseminor, unsigned count, const char* name);`
      - Valor escolhido fica armazenado em `dev`

14

14

# MAJOR E MINOR NUMBERS

- 12 bits para *major number* e 20 bits para *minor number*
- Tipo de dados: **dev\_t**
- **Macros:**
  - Para compor número de dispositivo: MKDEV(int major, int minor);
  - Para extrair *minor* e *major numbers*:
    - MAJOR(dev\_t dev);
    - MINOR(dev\_t dev);
- **Exemplo:**
  - static dev\_t my\_dev = MKDEV(202, 0);

Jorge Estrela da Silva

15

15

# CHARACTER DEVICES: REGISTO DAS FUNÇÕES PARA CADA OPERAÇÃO

```
struct file_operations {
 struct module *owner;
 loff_t (*llseek) (struct file *, loff_t, int);
 ssize_t (*read) (struct file *, char __user *, size_t, loff_t *);
 ssize_t (*write) (struct file *, const char __user *, size_t, loff_t *);
 long (*unlocked_ioctl) (struct file *, unsigned int, unsigned long);
 int (*mmap) (struct file *, struct vm_area_struct *);
 int (*open) (struct inode *, struct file *);
 int (*release) (struct inode *, struct file *);
};

//Initializes cdev, making it ready to add to the system with cdev_add().
void cdev_init(struct cdev *cdev, const struct file_operations *fops);

//Adds the device represented by p to the system, making it live immediately.
int cdev_add(struct cdev *p, dev_t dev, unsigned count);
```

Jorge Estrela da Silva

16

16

# CHARACTER DRIVER: EXEMPLO

```
const struct file_operations my_fops = {
 .owner = THIS_MODULE,
 .open = my_open,
 .read = my_read,
 .write = my_write,
 .release = my_release,
};
static struct cdev mycdev;

int init_module(void)
{
 dev_t devno = MKDEV(MY_MAJOR, 0);
 register_chrdev_region(devno, 1, "my_device_driver"); //1 significa apenas 1 minor number
 cdev_init(&mycdev, &my_fops);
 cdev_add(&mycdev, devno, 1); // 1 significa que fica associado ao minor number 1

 return 0;
}
```

Jorge Estrela da Silva

17

17

# CHARACTER DRIVER: EXEMPLO (CONT.)

```
//Chamada quando ficheiro de dispositivo é lido (e.g., cat /dev/isep_chrdev)
ssize_t my_read (struct file *f, __user char *buf, size_t sz, loff_t *off) {
 pr_info("my_read was called.\n");
 return 0;
}
```

- Pedido de leitura de até **sz** bytes, a partir da “posição” **\*off**
- Parâmetro **buf** deverá conter um apontador para uma região de memória do espaço do processo, com espaço suficiente para receber **sz** bytes
- Deverá retornar número de bytes enviados para o processo
  - i.e., que serão **lidos** pelo processo
  - Atualizar **\*off** com número de bytes enviados

Jorge Estrela da Silva

18

18

## CHARACTER DRIVER: EXEMPLO (CONT.)

```
//Chamada quando é realizado uma operação de escrita de dados no ficheiro de dispositivo
static int my_write (struct file *f, __user const char *buf, size_t sz, loff_t *off) {
 pr_info("my_write was called.\n");
 return 0;
}
```

- Pedido de escrita de **sz** bytes, a partir da “posição” **\*off**
- Parâmetro **buf** deverá conter um apontador para uma região de memória do espaço do processo contendo os **sz** bytes a serem recebidos pelo LKM
- Deverá retornar número de bytes recebidos pelo LKM
  - i.e., que serão considerados **escritos** pelo processo
  - Atualizar **\*off** com número de bytes recebidos

Jorge Estrela da Silva

19

19

## TRANSFERÊNCIA DE DADOS ENTRE ESPAÇO DE UTILIZADOR E ESPAÇO DO KERNEL

- Cópias de dados entre endereços do processo e endereços do kernel têm que ser feitos recorrendo a funções **copy\_to\_user** e **copy\_from\_user**
  - Porquê?

Jorge Estrela da Silva

20

20

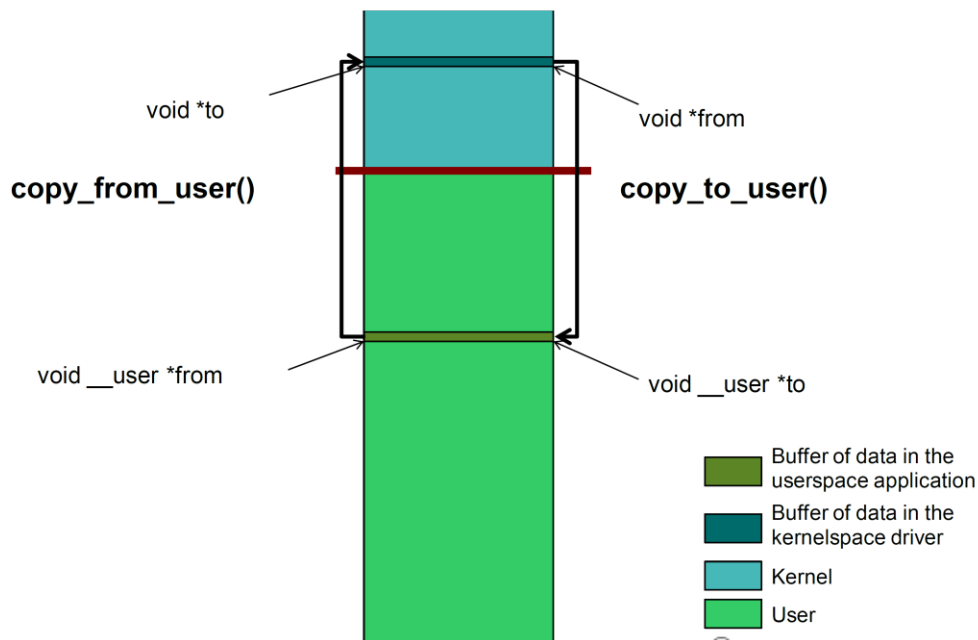
# TRANSFERÊNCIA DE DADOS ENTRE ESPAÇO DE UTILIZADOR E ESPAÇO DO KERNEL

- Linux usa mecanismo de paginação:
  - Memória alocada pelos programas (*malloc*, *stack*, etc.) pode não estar ainda fisicamente alojada na memória principal.
    - Em geral, isso só ocorre quando o programa escreve dados nas variáveis ou blocos de memória alocados
  - Páginas de memória do processo podem ter sido transferidas para “*swap*” (memória virtual) para libertar RAM para outros processos.
- Em ambos os casos mencionados acima, um acesso às páginas não presentes em RAM vai gerar uma exceção (*page fault*).
  - É normal acontecer em processos do utilizador. kernel seleciona automaticamente uma “moldura” de página (*page frame*) na RAM para poder carregar os dados em falta.
  - Quando o código está a correr em modo de kernel, é necessário controlar a ocorrência das *page faults*.
- `copy_to_user`** e **`copy_from_user`** fazem também verificação de endereços inválidos

Jorge Estrela da Silva

21

21



22

22

# TRANSFERÊNCIA DE DADOS ENTRE ESPAÇO DE UTILIZADOR E ESPAÇO DO KERNEL

- `unsigned long copy_to_user(void __user *to, const void *from, unsigned long n);`
- `unsigned long copy_from_user(void *to, const void __user *from, unsigned long n);`
- The return value must be checked. Zero on success, non-zero on failure.
  - If non-zero, the convention is to return `-EFAULT`.
- Other functions:
  - `get_user(kernelv, p)` - kernel variable **kernelv** gets the value pointed by the user-space pointer **p**
  - `put_user(kernelv, p)` - value pointed by the user-space pointer **p** is set to the contents of the kernel variable **kernelv**

Jorge Estrela da Silva

23

23

# EXEMPLO DE FUNÇÃO DE LEITURA

```
static ssize_t
my_read(struct file *file, char __user *buf, size_t count, loff_t *off) {
 int remaining_size, transfer_size;
 remaining_size = available_bytes - (int) (*off); // bytes left to transfer
 if (remaining_size == 0) /* All read, returning 0 (End Of File) */
 return 0;

 /* Size of this transfer */
 transfer_size = min(remaining_size, (int) count);
 if (copy_to_user(buf, mybuf + *off, transfer_size)) {
 return -EFAULT;
 } else { /* Increase the position in the open file */
 *off += transfer_size;
 return transfer_size;
 }
}
```

Jorge Estrela da Silva

24

24

# CRIAÇÃO DE FICHEIROS DE DISPOSITIVO

- Criação pelo próprio módulo:
  - `struct device * device_create(struct class * class, struct device * parent, dev_t devt, void * drvdata, const char * fmt, ...);`
    - Parâmetro **class** - pointer to the struct class that this device should be registered to
    - Parâmetro **parent** - pointer to the parent struct device of this new device, if any
    - Parâmetro **devt** - the `dev_t` for the char device to be added (major and minor numbers)
    - Parâmetro **drvdata** - data to be added to the device for callbacks
    - Parâmetro **fmt** - string for the device's name
      - ... - variable arguments
- Criação na linha de comando: **mknod**
  - Dispositivos orientados ao carácter: `mknod c major minor filename`
  - Dispositivos orientados ao bloco: `mknod b major minor filename`

Jorge Estrela da Silva

25

25

# CRIAÇÃO DE FICHEIROS DE DISPOSITIVO

- Criação pelo próprio módulo
  - Exemplo de utilização de **device\_create**:

```
struct class *mydevclass;
dev_t dev_num;
alloc_chrdev_region(&dev_num, 0, 1, "isep_lkm1");
arcom_class = class_create(THIS_MODULE, "isep_linuxdev");

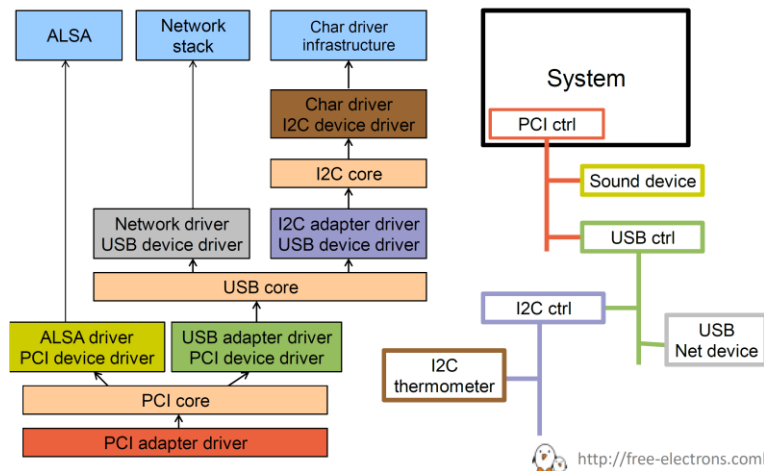
device_create(mydevclass,
 NULL, /* no parent device */
 dev_num,
 NULL, /* no additional data */
 "isep_chrdev");
```

Jorge Estrela da Silva

26

26

# CLASSES DE DISPOSITIVOS



27

## COMUNICAÇÃO ENTRE LKM E ESPAÇO DO UTILIZADOR

- Ficheiro de dispositivo (tipicamente em **/dev**)
- Através de “parâmetros”
  - Acessíveis na forma de ficheiros em **/sys/module/<module\_name>/parameters/**:  

```
#include <linux/moduleparam.h>
module_param(name, /* nome de variável previamente definida */
type, /* tipo de dados, verificado na altura da compilação: byte,
short, ushort, int, uint, long, ulong, charp, bool */
perm /* permissões do ficheiro */
);
```
  - Exemplo:  

```
int opmode=5; module_param(opmode, int, S_IRUGO); //permissões de leitura para todos
```
- Através de ficheiros em **/proc**

Jorge Estrela da Silva

28

28

# REGRAS DE DESENVOLVIMENTO

- Não esquecer:
  - Um LKM não é executado como um processo de utilizador.
  - É código que é carregado para o espaço do kernel
  - Funções são chamadas em resposta a ações dos processos dos utilizadores ou interrupções de hardware (incluindo temporizações)
    - A função **init\_module** é executada quando o módulo é carregado
- Funções podem ser acedidas por outros LKM se forem exportadas
  - **EXPORT\_SYMBOL(fn)** – fn pode ser acedida por qualquer módulos
  - **EXPORT\_SYMBOL\_GPL(fn)** – fn pode ser acedida apenas por módulos com licença GPL
- Atualmente, o kernel pode ser configurado para exigir que os módulos sejam assinados digitalmente
  - “module verification failed: signature and/or required key missing - tainting kernel”

Jorge Estrela da Silva

29

29

# REGRAS DE DESENVOLVIMENTO

- Código do kernel não pode usar bibliotecas do espaço do utilizador (/lib, etc.)
  - Incluindo a biblioteca standard do C !
  - Kernel tem a sua própria interface de programação. Exemplos:
    - **printk** em vez de **printf**
      - Mensagens para *buffer* de mensagens do kernel (volátil, consulta com **dmesg**), ficheiro de *log* (não volátil) ou consola principal (possível filtrar quais as mensagens a apresentar)
    - "void \* **memcpy**(void \*dest, const void \*src, size\_t count)"
      - Parâmetro *count* indica o número de elementos de 64 bit (e não bytes)
- Documentação da interface de programação do kernel:
  - <https://www.kernel.org/doc/html/docs/kernel-api/>
  - <https://www.kernel.org/doc/html/v5.10/core-api/kernel-api.html>

Jorge Estrela da Silva

30

30

# REGRAS DE DESENVOLVIMENTO

- Manutenção do código do módulo
  - Fora da árvore do *kernel*, compilado independentemente
    - Solução mais simples, apenas os ficheiros C e Makefile
    - × Obriga a compilar o módulo separadamente (operação adicional)
    - × Não é possível integrar estaticamente o código do módulo no ficheiro principal do kernel (*bzImage*)
    - × Kernel marca os módulos como menos seguros
      - × “loading out-of-tree module taints kernel”
  - Na árvore oficial do *kernel*
    - × Necessárias configurações adicionais nos scripts
    - Pode aparecer nas opções de compilação do *kernel*
    - Gerado juntamente com a imagem do kernel e restantes ficheiros ko.
      - Não é necessário compilar à parte
    - Só desta forma é possível compilar estaticamente

Jorge Estrela da Silva

31

31

# VERSÕES DE MÓDULOS E KERNEL

- Os módulos são sempre compilados para uma *release* (versão+configuração) específica do *kernel*
  - Necessário acesso aos ficheiros de cabeçalho e scripts de compilação
    - No Fedora: `sudo dnf install kernel-devel`
  - Também é possível usar código fonte de completo, que contém todos os ficheiros mencionados acima
- Utilização dos módulos
  - Não é possível carregar os módulos em sistemas com um *kernel* de versão diferente
    - “insmod: ERROR: could not insert module XXX: **Invalid module format**”
  - Não é possível carregar os módulos em sistemas com um *kernel* da mesma versão com configuração significativamente diferente da usada na compilação do módulo

Jorge Estrela da Silva

32

32

# PRINTK

- Possível definir o nível de urgência da mensagem (*default* é 4, KERN\_WARNING)
  - Exemplo: `printk(KERN_INFO "restaring\n");`
- Controlo de impressão de mensagens na consola
  - `cat /proc/sys/kernel/printk`
  - 4 4 1 7
    - The current (console) log level
      - All messages less than this value will appear on the console device!
    - The default level for messages that lack an explicit log level
    - The minimum allowed log level
    - The boot-time default log level

Jorge Estrela da Silva

33

33

# PRINTK E MACROS PR\_XXX

- `#define pr_emerg(fmt, ...) \`  
`printk(KERN_EMERG pr_fmt(fmt), ##__VA_ARGS__)`      */\* 0 - system is unusable \*/*
- `#define pr_alert(fmt, ...) \`  
`printk(KERN_ALERT pr_fmt(fmt), ##__VA_ARGS__)`      */\* 1 - action must be taken immediately \*/*
- `#define pr_crit(fmt, ...) \`  
`printk(KERN_CRIT pr_fmt(fmt), ##__VA_ARGS__)`      */\* 2 - critical conditions \*/*
- `#define pr_err(fmt, ...) \`  
`printk(KERN_ERR pr_fmt(fmt), ##__VA_ARGS__)`      */\* 3 - error conditions \*/*
- `#define pr_warning(fmt, ...) \`  
`printk(KERN_WARNING pr_fmt(fmt), ##__VA_ARGS__)`      */\* 4 - warning conditions \*/*
- `#define pr_warn pr_warning`
- `#define pr_notice(fmt, ...) \`  
`printk(KERN_NOTICE pr_fmt(fmt), ##__VA_ARGS__)`      */\* 5 - normal but significant condition \*/*
- `#define pr_info(fmt, ...) \`  
`printk(KERN_INFO pr_fmt(fmt), ##__VA_ARGS__)`      */\* 6- informational \*/*
- KERN\_DEBUG

Jorge Estrela da Silva

34

34

# REFERENCES

- <https://www.kernel.org/doc/html/v6.12/>

Jorge Estrela da Silva

